

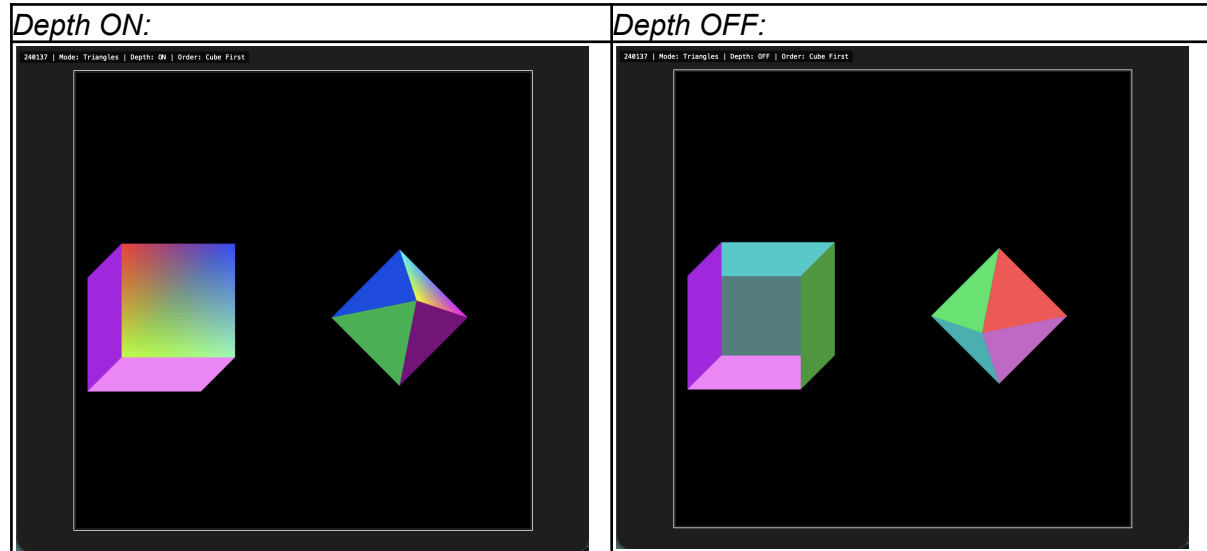
Write-up

Student: Margarita Serebrennikova

Student ID: 240137

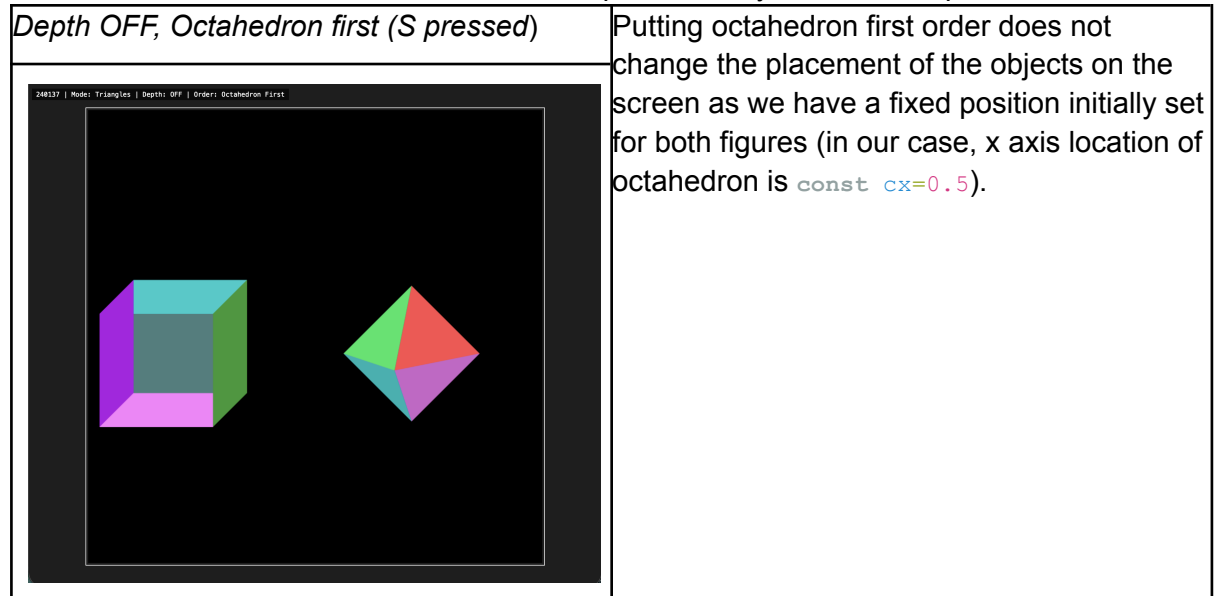
Octahedron, Offset (-0.15; -0.15)

E1 — Depth test vs draw order



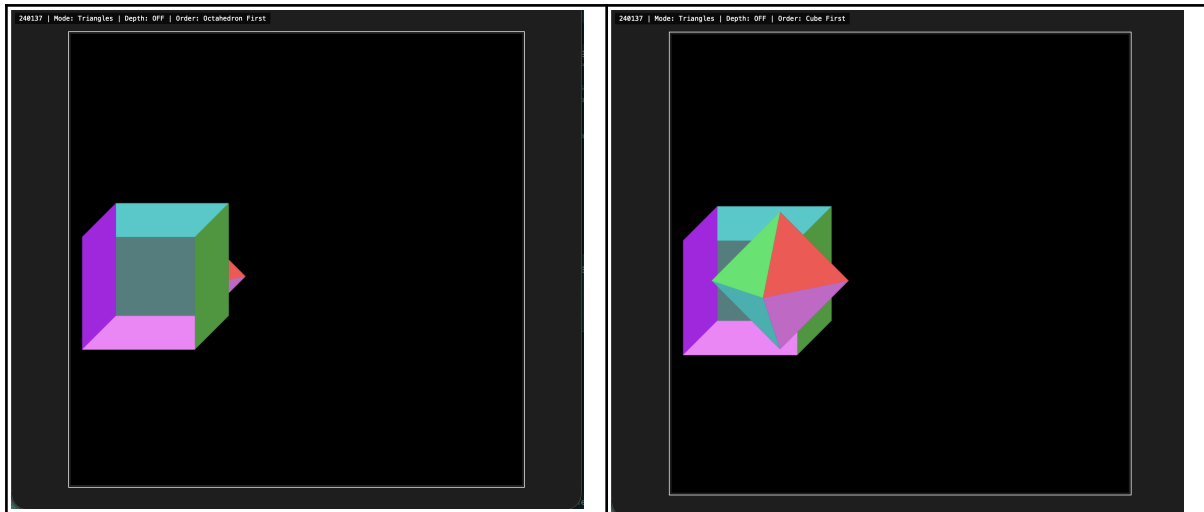
With the **depth ON**, I only see the front (gradient), left side (violet) and bottom (pink) of the cube, while with **depth OFF**, I can see the backside (gray), left (violet) & right (green) sides, top (teal blue) and bottom (pink).

When we turn off the depth, last face drawn, which in code is right side, comes to the front as every fragment overwrites the pixel, while the initially written as first, which is front side, comes to last in order when we turn off the depth, so every other side is placed above it.



For example, if we put a different location of `cx` in `index.js`, we then change current location of the octahedron so that it overlaps with cube:





Now that we've changed the x axis position of the octahedron, we see that when we press S, order **draws the octahedron first**, which then puts the cube above it. While if we have a cube first order, the cube is drawn first and the octahedron gets located above the cube.

E2 — Why negative z is in front

Ch. 3 says z increases towards the viewer, yet the square at $z = -0.3$ is drawn in front of the one at $z = +0.3$. However, in our program, the coordinates go directly into clip space, with no camera or perspective, so positive z points into the screen and negative z points out towards us. Before we begin drawing, `gl.clear` with `DEPTH_BUFFER_BIT` fills the depth buffer with the default value 1.0, the farthest possible depth, and then GPU maps z into depth buffer with values between 0 and 1, so the square at $z = -0.3$ gets a smaller depth than the one at $z = +0.3$. According to `gl.LEQUAL` rule, a fragment is only drawn if the new fragment's depth is less than or equal to the depth already stored for that pixel. Front square passes because its depth is less than 1, and its depth is stored. The back square fails because now its value is greater than the depth that was already stored by front square, so it's hidden.

Predict and test two changes:

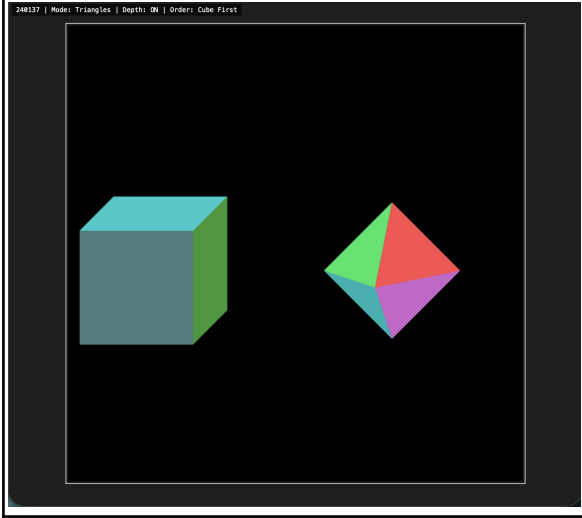
(a) `gl.depthFunc(gl.GREATER)` alone;

Prediction: Nothing will be seen if we change `gl.LEQUAL` to `gl.GREATER`, cause there are no fragments with depth more than 1

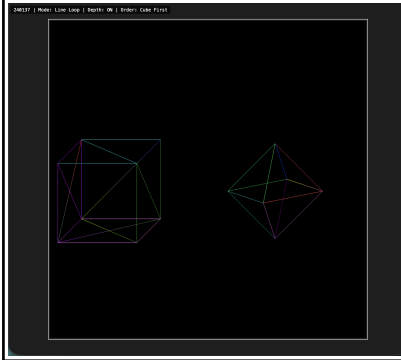
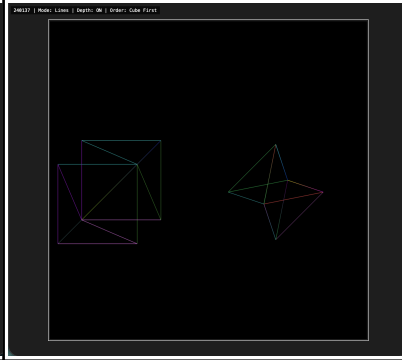
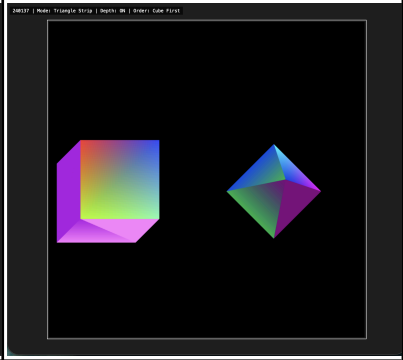
Test:	Observations:
	Prediction was correct

(b) `gl.GREATER` together with `gl.clearDepth(0.0)`.

Prediction: Now if we set depth parameter to 0 instead of 1, there are parameters of depth larger than 0, which makes these sides visible

Test:	Observations:
	My prediction was true, but additionally we can highlight that we see the furthest sides of both objects

E3 — Drawing modes on triangle data

Mode 2	Mode 3	Mode 6
		

Each face is stored as a,b,c, a,c,d

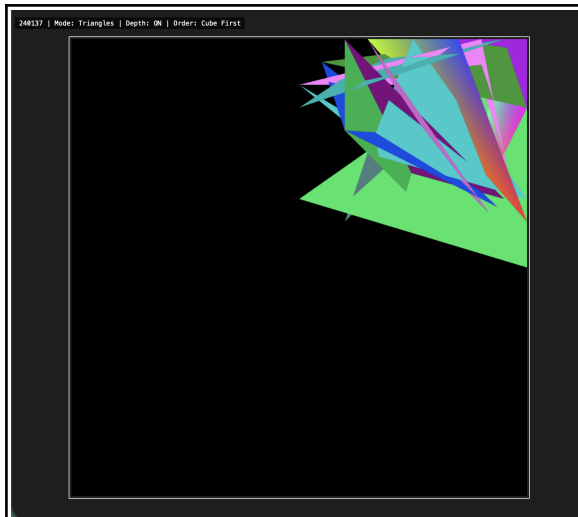
In LINE_LOOP: every vertex is joined with the next one, so 0->1, 1->2, 2->3....34->35 AND the last one (35) is **connected back** to vertex 0, because they form a loop. It creates extra line.

In LINES: we take vertex pairs a->b, c->a and c->d. This makes 2 edges and diagonal. And since in order a b c a c d bc and da are not next to each other, they are not drawn.

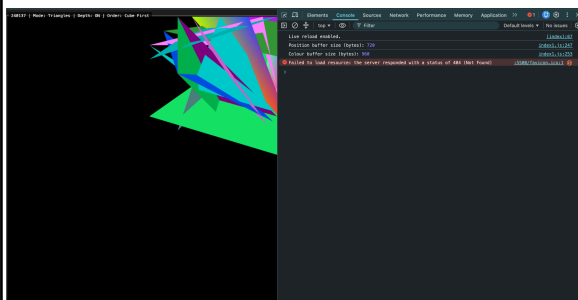
In TRIANGLE_STRIP: this mode forms triangles from vertex and 2 previous vertices, but our array is ordered in independent groups of 3 vertices to form triangles, so connecting with previous vertices just creates extra triangles and instead of 12 triangles to form cube for instance, we have 34 triangles, thus some duplicate or degenerate with some joining corners of neighboring faces creating incorrect triangles cutting.

E4 — The missing bind

Removing line	Explanation:
<pre>gl.bindBuffer(gl.ARRAY_BUFFER, buffers.position);</pre>	



Console shows no errors:



All the triangles making up objects ended up toggling to each other on the upper right corner of the canvas.

Every buffer stores particular information, and in our case we removed the line that binds the position buffer, and in order, `initBuffers` fills up the position buffer before the color buffer, so after `initBuffers` finishes, the colour buffer is still the one bound.

Without that bind, `gl.VertexAttribPointer(...)` connects `aPosition` to whichever buffer is currently bound. So now, the `rgba` data from the color buffer is perceived as the position of vertices. The position buffer is never connected to `aPosition`, so its data is never read. Colour values are between 0 and 1, so every `x` and `y` is ≥ 0 , which places all vertices in the upper-right quarter.

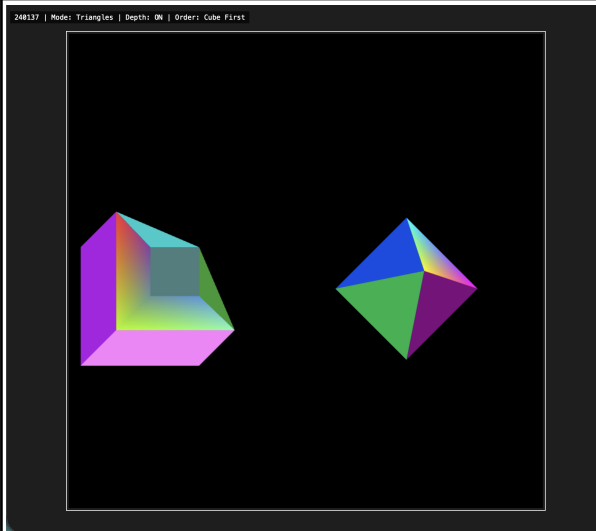
Program doesn't give error because WebGL only ensures that buffer has enough float data for vertices, and in our case we have $(36+24)$ vertices $\times 3$ floats $\times 4$ bytes = 720 bytes.

Meanwhile, in the color buffer we have 960 bytes as shown in the console. Thus, since $720 < 960$, it's enough bytes to not produce any error.

E5 — Clipping

Change `z=1.2`:

`ftr: [R, T, F] -> ftr: [R, T, 1.2]`



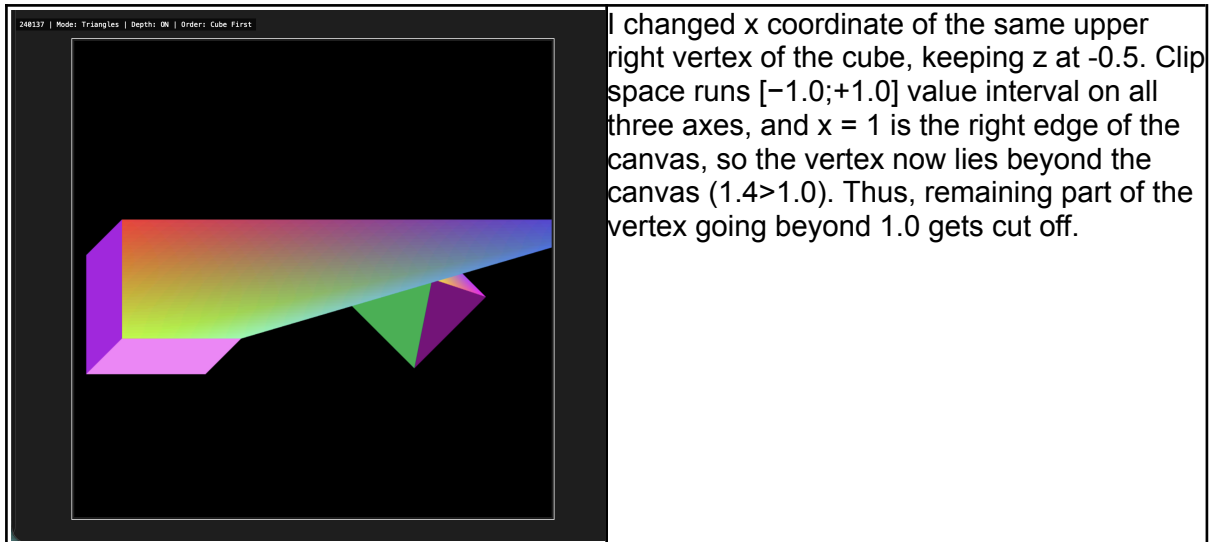
Observation:

Front top right corner of cube is cut off, because `z` is beyond far edge of clip space (where max is 1.0, and `z` right now is 1.2) GPU therefore only keeps a part that fits max limit of $[-1.0; 1.0]$, cutting off everything exceeding this interval.

Change `x=1.4`:

`ftr: [R, T, F] -> ftr: [1.4, T, F]`

Observation:

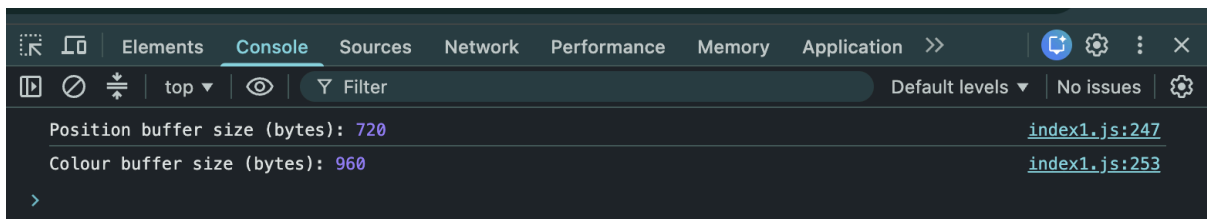


E6 — Buffer sizes

Vertex count $N=24+36=60$

Buffer size Positions $=60*3*4=720$ bytes

Buffer size Colors $=60*4*4=960$ bytes



Development Log:

Bug	Fix
in vsSource and fsSource forgot to add backticks <pre> /*===== Shaders =====*/ const vsSource = attribute vec4 aPosition; attribute vec4 aVertexColor; varying lowp vec4 vColor; void main() { gl_Position = aPosition; gl_PointSize = 6.0; vColor = aVertexColor; } ; const fsSource = varying lowp vec4 vColor; void main() { gl_FragColor = vColor; } </pre>	Adding backticks turns code into string because vsSource and fsSource are not supported by JS so it got me a Syntax Error
Used different wording for attribute variables: First I stated <i>aVertexPosition</i> in shader, but then used <i>aPosition</i> name for attribute, so <i>aPosition</i> wasn't defined and my code didn't run	Ensured that I use same attribute names consistently
Mistyped Octahedron's first face [nx, px, py], so solid was closed and gradient went missing.	Instead, I should've written [nz, px, py]
Keys were case-sensitive, and render() ran on every key, so lowercase d and s didn't work.	Ensured that I typed in capital letters <pre> state.mode = KEY_TO_MODE[key]; } else if (key === "D") { state.depth = !state.depth; } else if (key === "S") { state.cubeFirst = !state.cubeFirst; } else { return; } render(); </pre>